```
Article.created_before(Time.zone.now)
```

However, this is just duplicating the functionality that would be provided to you by a class method.

```
class Article < ActiveRecord::Base
  def self.created_before(time)
    where("created_at < ?", time)
  end
end
```

Using a class method is the preferred way to accept arguments for scopes. These methods will still be accessible on the association objects:

```
category.articles.created_before(time)
```

14.2 Applying a default scope

If we wish for a scope to be applied across all queries to the model we can use the `default_scope`method within the model itself.

```
class Client < ActiveRecord::Base
  default_scope { where("removed_at IS NULL") }
end
```

When queries are executed on this model, the SQL query will now look something like this:

```
SELECT * FROM clients WHERE removed_at IS NULL
```

If you need to do more complex things with a default scope, you can alternatively define it as a class method:

```
class Client < ActiveRecord::Base
  def self.default_scope
    # Should return an ActiveRecord::Relation.
  end
end
```

14.3 Merging of scopes

Just like `where` clauses scopes are merged using `AND` conditions.

```
class User < ActiveRecord::Base
  scope :active, -> { where state: 'active' }
  scope :inactive, -> { where state: 'inactive' }
end

User.active.inactive
# SELECT "users".* FROM "users" WHERE "users"."state" = 'active' AND
"users"."state" = 'inactive'
```

We can mix and match `scope` and `where` conditions and the final sql will have all conditions joined with `AND`.

```
User.active.where(state: 'finished')
# SELECT "users".* FROM "users" WHERE "users"."state" = 'active' AND
"users"."state" = 'finished'
```

If we do want the last `where` clause to win then `Relation#merge` can be used.

```
User.active.merge(User.inactive)
# SELECT "users".* FROM "users" WHERE "users"."state" = 'inactive'
```

One important caveat is that `default_scope` will be prepended in `scope` and `where` conditions.

```
class User < ActiveRecord::Base
  default_scope { where state: 'pending' }
  scope :active, -> { where state: 'active' }
  scope :inactive, -> { where state: 'inactive' }
end
```